

ACP Architecture

Agent - Control - Process

Project: AI Chat Backend

Version: 2.1.0

Stack: Python 3.12 + FastAPI + aiomysql + JSON-RPC 2.0

Date: 2026-07-15

Table of Contents

1. Directory Structure
2. Call Chain: User Sends Message
 - 2.1 HTTP Route
 - 2.2 Process Orchestration
 - 2.3 ACP Client
 - 2.4 ACP Server
 - 2.5 Handler
 - 2.6 Provider
3. Skill Execution Chain
4. Workflow Execution Chain
5. Provider Registration & Routing
6. Plugin Middleware System
7. Supporting Systems
 - 7.1 Memory
 - 7.2 RAG
 - 7.3 Cache
 - 7.4 Event
 - 7.5 Eval
 - 7.6 MCP
8. Startup Flow
9. Protocol Definition

1. Directory Structure

server/python_app/

```

├── acp/ ACP Core Layer
│   ├── protocol.py JSON-RPC 2.0 Protocol
│   ├── client.py ACP Client (in-process)
│   ├── server.py ACP Server (routing + plugins)
│   ├── router.py Route table (internal/external)
│   ├── session.py Session management
│   ├── middleware/ Plugin System
│   │   ├── base.py Plugin abstract base
│   │   ├── registry.py Plugin registry (chain call)
│   │   ├── logging.py Request logging
│   │   ├── rate_limit.py Rate limiting
│   │   ├── auth.py Auth passthrough
│   │   ├── prompt_injector.py Prompt injection
│   │   ├── sensitive_filter.py Sensitive word filter
│   │   └── response_formatter.py Response formatting
│   ├── handlers/ Request handlers
│   │   ├── chat.py chat.completions
│   │   ├── models.py models.list
│   │   ├── health.py health.check
│   │   ├── skills.py skills.list/execute
│   │   └── workflow.py workflow.execute
│   ├── skills/ Skill System
│   │   ├── base.py Skill abstract base
│   │   ├── registry.py Registry + executor
│   │   ├── loader.py Hot loader
│   │   └── builtin/ 7 builtin skills
│   ├── workflow/ Workflow Engine
│   │   ├── engine.py Engine
│   │   └── workflows/ 4 preset YAML workflows
│   ├── memory/ Memory System
│   ├── rag/ RAG Knowledge Base
│   ├── mcp/ MCP Protocol
│   ├── cache/ Cache System
│   ├── events/ Event System
│   ├── eval/ Evaluation System
│   └── external/ External ACP Agents

├── agent/ Provider Implementations
│   ├── base.py Provider abstract
│   ├── registry.py Provider registry
│   ├── opencode.py OpenCodeProvider
│   └── deepseek.py DeepSeekProvider

```

```
├─ control/auth.py API Key Management
├─ process/chat.py Session Orchestration
├─ routes/ Route Layer
| ├─ chat.py /api/chat/*
| ├─ skills.py /api/skills/*
| ├─ workflows.py /api/workflows/*
| └─ admin.py /api/admin/*
├─ config/ 9 YAML configs
├─ main.py Application Entry
├─ cli.py CLI Tool
└─ public/ Frontend Pages
```

2. Call Chain: User Sends Message

User types text - clicks send

- => POST /auth/api/chat/sessions/{sid}/send {content: "hello"}
- => routes/chat.py (FastAPI + JWT auth)
- => process/chat.py (save msg - get session - history - API key)
- => ACPClient.call("chat.completions", params)
- => ACPServer.route_request() (plugins - router - handler)
- => ChatHandler - Provider.send()
- => OpenCodeProvider / DeepSeekProvider - AI API
- => Response back through all layers to frontend

2.1 HTTP Route (routes/chat.py)

Responsibility: Receive HTTP requests, parse parameters, delegate to Process layer.
No AI logic here.

```
# routes/chat.py - Thin routing layer

router = APIRouter(prefix="/api/chat", tags=["chat"])

# JWT auth is injected via Depends(get_current_user)

@router.post("/sessions/{sid}/send")
async def send_to_ai(
    sid: str,
    req: SendReq,
    user: dict = Depends(get_current_user),
):
    # Delegate entirely to Process layer
    result = await send_message(
        sid,                # session ID from URL
        user["id"],         # user ID (for API key lookup)
        user["username"],   # username (for DB queries)
        req.content,        # the user's message text
    )
    return result
```

2.2 Process Orchestration (process/chat.py)

Responsibility: Core orchestration layer: save message, get session info, get history, get API key, call ACP client, save AI response.

```
# process/chat.py - Core orchestration

_acp_client: Optional[ACPClient] = None

def set_acp_client(client: ACPClient):
    global _acp_client
    _acp_client = client # injected by main.py at startup

async def send_message(sid, user_id, username, content):
    # 1. Save user message to MySQL
    await execute_write(
        "INSERT INTO chat_messages (...) VALUES (%s,%s,%s)",
        (sid, "user", content),
    )

    # 2. Get session info (model + oc_session_id)
    session = await execute_one(
        "SELECT oc_session_id, model FROM chat_sessions WHERE id=%s", (sid,),
    )
    session_model = session["model"] if session else ""
    if not session_model:
        return {"success": False, "error": "No model selected"}

    # 3. Find provider by model ID
    provider_inst = _get_provider_for_model(session_model)
    # Returns OpenCodeProvider() or DeepSeekProvider()

    # 4. Get conversation history
    prev = await execute("SELECT role, content FROM chat_messages WHERE ...")

    # 5. Get API key if needed (DeepSeek requires key, OpenCode does not)
    if registry.get_config(provider_inst.provider_id).get("requires_key"):
        user_keys = await get_user_api_keys(user_id)
        for sk, sv in cfg.get("sub_providers", {}).items():
            if user_keys.get(sv.get("key_type")):
                api_key = user_keys[sv["key_type"]]
                break
        if not api_key:
            return {"success": False, "error": "API key not configured"}

    # 6. OpenCode needs remote session management
    if provider_inst.provider_id == "opencode":
        if not oc_id:
            oc = await provider_inst.create_session()
            oc_id = oc.get("id")
            await provider_inst.set_model(oc_id, session_model)
```

```

# 7. Send via ACP protocol
if _acp_client:
    # --> Enter ACP layer -->
    acp_result = await _acp_client.call(ACPMMethod.CHAT_COMPLETIONS, {
        "provider": provider_inst.provider_id,
        "model": session_model,
        "messages": prev,
        "user_content": content,
        "api_key": api_key,
        "oc_session_id": oc_id,
    })

# 8. Save AI response and return
await execute_write(
    "INSERT INTO chat_messages (...) VALUES (%s,%s,%s,%s)",
    (sid, "assistant", acp_result["content"], acp_result.get("thinking", "")),
)
return {"success": True, "content": acp_result["content"], "thinking": ...}

```

2.3 ACP Client (acp/client.py)

Responsibility: Encapsulate JSON-RPC 2.0 requests. In-process direct call to ACPServer (zero network overhead).

```

# acp/client.py

class ACPClient:
    def __init__(self, server: ACPServer = None):
        self._server = server # injected by main.py
        self._counter = 0 # auto-increment request ID

    async def call(self, method: str, params: dict) -> dict:
        self._counter += 1
        # 1. Build JSON-RPC 2.0 request
        req = ACPRequest(method=method, params=params, id=self._counter)

        # 2. In-process direct call (no HTTP)
        resp: ACPResponse = await self._server.route_request(req)

        # 3. Unpack response
        if resp.error:
            return {"success": False, "error": resp.error.get("message", "ACP error")}
        return resp.result

```

2.4 ACP Server (acp/server.py)

Responsibility: Receive ACPRequest, process through plugin chain (pre) - Router - Handler - plugin chain (post).

```
# acp/server.py

class ACPServer:
    def __init__(self, external_gateway=None):
        self.router = Router()
        self._handlers = {}
        self._register_default_handlers()

    def _register_default_handlers(self):
        self._handlers["chat.completions"] = ChatHandler()
        self._handlers["models.list"] = ModelsHandler()
        self._handlers["health.check"] = HealthHandler()
        self._handlers["skills.list"] = SkillsHandler()
        self._handlers["skills.execute"] = SkillsHandler()
        self._handlers["workflow.execute"] = WorkflowHandler()

    async def route_request(self, request: ACPRequest) -> ACPResponse:
        # === Pre-plugin chain (logging -> auth -> rate_limit -> ...) ===
        blocked = await plugin_registry.run_on_request(method, params)
        if blocked:
            return ACPResponse.fail(ACPErrorCode.RATE_LIMIT, str(blocked))

        # === Router dispatch ===
        # internal -> local Handler, external -> ExternalGateway
        if params.get("provider") and self.router.is_external(params["provider"]):
            return await self.external_gateway.call(...)

        # === Find and run handler ===
        handler = self._handlers.get(method)
        result = await handler(params)

        # === Post-plugin chain ===
        resp.result = await plugin_registry.run_on_response(method, params,
        resp.result)
        return resp
```

2.5 Handler (acp/handlers/chat.py)

Responsibility: Bridge between ACP protocol and Provider.send().

```
# acp/handlers/chat.py

class ChatHandler:
```



```

async def __call__(self, params: dict) -> ACPResponse:
    # 1. Get provider instance from registry
    provider_inst = registry.get(params["provider"])
    # registry.get("opencode") -> OpenCodeProvider instance

    # 2. Call Provider.send() - each provider implements its own
    result: SendResult = await provider_inst.send(
        model=params.get("model"),
        messages=params.get("messages", []),
        user_content=params.get("user_content"),
        api_key=params.get("api_key", ""),
        oc_session_id=params.get("oc_session_id", ""),
    )

    # 3. Return standardized ACP response
    return ACPResponse.success({
        "content": result.content,
        "thinking": result.thinking,
        "ok": result.ok,
        "error": result.error,
    })

```

2.6 Provider (agent/opencode.py)

```

# agent/opencode.py - OpenCodeProvider implementation

class OpenCodeProvider(Provider):
    provider_id = "opencode"

    async def send(self, model, messages, user_content, **kwargs) -> SendResult:
        oc_id = kwargs.get("oc_session_id", "")

        # 1. Create remote opencode session if needed
        if not oc_id:
            oc = await self.create_session()          # POST /api/session
            oc_id = oc.get("id")
            await self.set_model(oc_id, model)        # POST /api/session/{id}/model

        # 2. Send prompt
        await self.send_prompt(oc_id, user_content, model)

        # 3. Poll for response (up to 120s)
        full_text = ""
        async for chunk in self.poll_response(oc_id, timeout=120):
            if chunk.get("done"): break
            full_text += chunk.get("content", "")

        # 4. Parse <think> tags into content/thinking
        thinking_part, clean_text = self._parse_thinking(full_text)
        return SendResult(content=clean_text, thinking=thinking_part)

```

Complete Call Sequence

Frontend

| POST /api/chat/sessions/{sid}/send

v

routes/chat.py

| JWT auth - send_message(sid, user_id, username, content)

v

process/chat.py

| 1. INSERT chat_messages (save user msg)
 | 2. SELECT model FROM chat_sessions (get model)
 | 3. _get_provider_for_model(model) -> OpenCodeProvider
 | 4. SELECT role,content FROM chat_messages (history)
 | 5. get_user_api_keys(user_id) (API keys)
 | 6. ACPClient.call("chat.completions", {...})

v

acp/client.py

| ACPRequest(method, params, id) -> route_request()

v

acp/server.py

| plugin_registry.run_on_request() -> all pass
 | router.is_external("opencode") -> False -> internal
 | handler = _handlers["chat.completions"] = ChatHandler

v

acp/handlers/chat.py

| registry.get("opencode") -> OpenCodeProvider instance
 | provider.send(model, messages, user_content, ...)

v

agent/opencode.py

| POST /api/session -> create remote session
 | POST /api/session/{id}/model -> set model
 | POST /api/session/{id}/prompt -> send prompt
 | GET /api/session/{id}/message (poll) -> get response

v

Response Chain

| SendResult -> ACPResponse.success()
 | -> plugin_registry.run_on_response()
 | -> ACPClient.call() returns dict
 | -> INSERT chat_messages (save AI reply)
 | -> JSONResponse -> frontend renders

3. Skill Execution Chain

Entry Points

Option 1 - REST API: `POST /api/skills/{name}/execute {args: {...}}`

```
# routes/skills.py
@router.post("/{name}/execute")
async def execute_skill(name, body, user = Depends(get_current_user)):
    return await skill_registry.execute(name, **body.get("args", {}))
```

Option 2 - ACP RPC: `POST /api/chat/acp/rpc` with

`{"method": "skills.execute", "params": {"skill": "web_search", "args": {...}}}`

```
# acp/handlers/skills.py
class SkillsHandler:
    async def __call__(self, params):
        action = params.get("action", "list")
        if action == "execute":
            result = await skill_registry.execute(params["skill"],
            **params.get("args", {}))
            return ACPResponse.success(result)
        # action == "list" -> return all registered skills
        return ACPResponse.success(skill_registry.list_all())
```

SkillRegistry Execution

```
# acp/skills/registry.py
async def execute(self, name, **kwargs):
    # Find skill instance by name
    skill = self._skills[name] # e.g. DocumentAnalysisSkill()
    # Call the skill's execute method
    result = await skill.execute(**kwargs)
    return {"success": True, "result": result}
```

doc_analyzer - Document Analysis Skill

```
class DocumentAnalysisSkill(Skill):
    name = "doc_analyzer"
    description = "Read product docs and analyze: summary / changelog / API / glossary"

    async def execute(self, path="", mode="auto"):
```

```

text = Path(path).read_text(encoding="utf-8")
lines = text.splitlines()

result = {"file": path, "size": len(text), "lines": len(lines)}

if mode in ("auto", "summary"):
    result["summary"] = self._summarize(text, lines)
if mode in ("auto", "changelog"):
    result["changelog"] = self._extract_changelog(lines)
if mode in ("auto", "api"):
    result["api"] = self._extract_api(lines)
if mode == "auto":
    result["toc"] = self._extract_toc(lines)
    result["keywords"] = self._extract_keywords(text)
return result

```

skill_generator - Self-Replicating Skill

```

class SkillGeneratorSkill(Skill):
    async def execute(self, requirement="", skill_name=""):
        # 1. Infer name from description
        name = self._make_name(requirement)
        # 2. Infer parameters by keyword matching
        params = self._infer_params(requirement)
        # 3. Infer imports from keywords
        imports = self._infer_imports(requirement)
        # 4. Generate Python code
        code = self._generate_code(class_name, name, params, imports)
        # 5. Write to skills/ directory
        Path("../..//skills") / name / "main.py").write_text(code)
        # 6. Hot-load into running process
        spec = importlib.util.spec_from_file_location(name, py_path)
        mod = importlib.util.module_from_spec(spec)
        spec.loader.exec_module(mod)
        skill_registry.register(mod.skill)
        return {"skill_name": name, "file": str(py_path), "code": code}

```

4. Workflow Execution Chain

```
API: POST /api/workflows/report_generator/execute {args: {topic: "AI"}}
=> routes/workflows.py -> WorkflowEngine.execute("report_generator",
topic="AI")
=> Read YAML nodes -> execute sequentially
=> node[0]: type=skill -> web_search(query="AI")
=> node[1]: type=llm -> prompt using results[0]
=> Return all node results
```

```
# YAML workflow definition
workflows:
  report_generator:
    nodes:
      - name: search_data
        type: skill
        skill: web_search
        args: {query: "{{input.topic}}"} # template variable
      - name: generate_report
        type: llm
        prompt: "Generate report from {{results[0]}}"

# acp/workflow/engine.py
class WorkflowEngine:
    async def execute(self, name, **input_args):
        wf = self._workflows[name]
        results = []
        for node in wf["nodes"]:
            if node["type"] == "skill":
                r = await self._skill_registry.execute(node["skill"], **node["args"])
                results.append(r)
            elif node["type"] == "http":
                async with httpx.AsyncClient() as c:
                    r = await c.request(node["method"], node["url"])
                    results.append({"status": r.status_code})
        return {"success": True, "results": results}
```

5. Provider Registration and Routing

```
# agent/registry.py - Two registration methods

# Method 1: Code registration (in main.py)
provider_registry.register(OpenCodeProvider)

# Method 2: YAML config loading
provider_registry.load_config("config/providers.yaml")
# Loads deepseek from YAML:
# {
#   "module": "python_app.agent.deepseek",
#   "class": "DeepSeekProvider",
#   "type": "internal",
#   "requires_key": true,
#   "models": [{"id": "deepseek-v4-pro", "name": "DeepSeek V4 Pro"}, ...]
# }

# Router maps provider_id -> route type
class Router:
    _routes = {
        "opencode": "internal", # -> local Handler
        "deepseek": "internal", # -> local Handler
        "hermes": "external",   # -> ExternalGateway
    }
```

6. Plugin Middleware System

```
# acp/middleware/base.py
class Plugin(abc.ABC):
    name: str = "base"
    priority: int = 100 # lower = runs first

    async def on_request(self, method, params) -> dict | None:
        """Pre-hook: return None to allow, dict to block"""
        return None

    async def on_response(self, method, params, result):
        """Post-hook: can modify result"""
        return result

# Execution order (by priority):
# Logging(10) -> Auth(15) -> RateLimit(20) -> SensitiveFilter(30)
# -> PromptInjector(50) -> Handler -> Logging.on_response
# -> ResponseFormatter(200)
```

7. Supporting Systems

7.1 Memory (acp/memory/manager.py)

```
class MemoryManager:
    short_term = ShortTermMemory() # in-memory dict, max 100
    long_term = LongTermMemory() # JSON file persistence
    working = WorkingMemory() # single-request scope
```

7.2 RAG (acp/rag/retriever.py)

```
class SimpleRAG:
    async def retrieve(self, query, top_k=5):
        # Keyword matching retrieval
        keywords = set(query.lower().split())
        scored = [(sum(kw in doc["content"].lower() for kw in keywords), doc)
                  for doc in self._docs]
        return [doc for _, doc in sorted(scored, reverse=True)[:top_k]]
```

7.3 Cache (acp/cache/base.py)

```
class SimpleCache:
    _store = {} # {key: (value, expiry)}
    async def get(self, key):
        entry = self._store.get(key)
        return entry[0] if entry and time.time() < entry[1] else None
```

Three instances: `PromptCache` (1h), `EmbeddingCache` (24h), `ResultCache` (10min)

7.4 Event (acp/events/emitter.py)

```
# Pub/sub pattern
emitter = EventEmitter()
emitter.on("chat.completions", logging_handler)
emitter.on("chat.completions", audit_handler)
await emitter.emit(ACPEvent("chat.completions", {"method": method, "params": params}))
```


7.5 Eval (acp/eval/runner.py)

```
# Dataset + metrics evaluation
runner = EvalRunner()
runner.load_dataset("qa_test", [{"input": ..., "expected": ...}])
result = await runner.run("accuracy", "qa_test")
# Returns {"category": "accuracy", "average_score": 0.95, "details": [...]}
```

7.6 MCP Client (acp/mcp/client.py)

```
# Call external MCP Server (e.g. hermes acp)
mcp = MCPClient(server_url="http://localhost:8080")
await mcp.call_tool("filesystem_read", {"path": "/tmp/test.txt"})
```

8. Startup Flow (main.py)

```
uvicorn python_app.main:app
=> FastAPI(lifespan=lifespan)
=> lifespan() - startup:
| 1. init_db() - MySQL connection pool
| 2. provider_registry.register(OpenCodeProvider)
| 3. provider_registry.load_config("config/providers.yaml")
| 4. skill_registry.register() x 7 (builtin skills)
| 5. load_skills_from_dir("server/skills/") (user skills)
| 6. plugin_registry.register() x 3 (middleware plugins)
| 7. Memory / RAG / Cache / Event / Eval initialization
| 8. ACPServer() + ACPClient(server) creation
| 9. WorkflowEngine() + load_dir() x 2
| 10. inject ACPClient into process/chat.py
=> on_event("startup"): inject acp_server into routes
=> Waiting for requests...
```

9. Protocol Definition (acp/protocol.py)

JSON-RPC 2.0 Request

```
{
  "jsonrpc": "2.0",
  "method": "chat.completions", // method name
  "params": {
    "provider": "opencode",
    "model": "deepseek-v4-flash-free",
    "messages": [...],
    "user_content": "hello",
    "api_key": "sk-..."
  },
  "id": 1
}
```

JSON-RPC 2.0 Response

```
{
  "jsonrpc": "2.0",
  "result": { // on success
    "content": "Hello!",
    "thinking": "",
    "ok": true,
    "error": ""
  },
  "error": null, // on failure: {"code": -32000, "message": "..."}
  "id": 1
}
```

Method Constants

Method	Purpose
chat.completions	AI Chat completion
models.list	List available models
health.check	Health check
skills.list / skills.execute	Skill operations

workflow.execute

Workflow execution

Error Codes

Code	Meaning
-32700	Parse error
-32601	Method not found
-32603	Internal error
-32000	Provider error
-32001	Rate limit
-32002	Auth error